



Recursion & Backtracking

Mathematical Thinking + State Space Exploration

Java 8+

THIRAVUGAL



Session Structure

1. Recursion fundamentals
2. Recursion tree & stack visualization
3. Tail recursion
4. Backtracking concept
5. Subset & permutation patterns
6. Constraint problems
7. Interview-level strategies



What is Recursion?

A function calling itself.

Requires:

1. Base case
2. Recursive case



Example — Factorial

```
static int factorial(int n) {  
    if (n == 0) return 1;  
    return n * factorial(n - 1);  
}
```

Time: $O(n)$

Space: $O(n)$



Call Stack Visualization

factorial(4)

$4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$

Stack builds $\rightarrow$ unwinds.



Fibonacci (Naive)

```
static int fib(int n) {  
    if (n <= 1) return n;  
    return fib(n - 1) + fib(n - 2);  
}
```

Time: $O(2^n)$

Exponential growth.



Recursion Tree Thinking

Each call branches.

Tree height = n

Nodes $\approx 2^n$

Understanding growth is critical.



Tail Recursion

Recursive call is last statement.

```
static int sum(int n, int acc) {  
    if (n == 0) return acc;  
    return sum(n - 1, acc + n);  
}
```

Java does NOT optimize tail recursion automatically.

Recursion vs Iteration

Feature	Recursion	Iteration
Stack usage	$O(n)$	$O(1)$
Clarity	Often clearer	Sometimes verbose
Risk	StackOverflow	Safer



When to Use Recursion?

- Tree problems
- Divide & conquer
- Backtracking
- DFS

THIRAVUGAL



What is Backtracking?

Backtracking =

Explore → Choose → Recurse → Undo (Backtrack)

Used for:

- Permutations
- Combinations
- Constraint problems
- Path finding

Subsets (Power Set)

Time: $O(2^n)$

Space: $O(n)$

```
static void subsets(int[] nums, int index,
    java.util.List<Integer> current) {

    System.out.println(current);

    for (int i = index; i < nums.length; i++) {
        current.add(nums[i]);
        subsets(nums, i + 1, current);
        current.remove(current.size() - 1);
    }
}
```



Permutations

Time: $O(n!)$

```
static void permute(int[] nums, int index) {
    if (index == nums.length) {
        System.out.println(java.util.Arrays.toString(nums));
        return;
    }

    for (int i = index; i < nums.length; i++) {
        swap(nums, i, index);
        permute(nums, index + 1);
        swap(nums, i, index);
    }
}

static void swap(int[] arr, int i, int j) {
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}
```



Combination Sum

Time: Exponential

```
static void combinationSum(int[] nums, int target,
    int index,
    java.util.List<Integer> current) {

    if (target == 0) {
        System.out.println(current);
        return;
    }

    if (target < 0) return;

    for (int i = index; i < nums.length; i++) {
        current.add(nums[i]);
        combinationSum(nums, target - nums[i], i, current);
        current.remove(current.size() - 1);
    }
}
```



N-Queens Problem

Place N queens such that:

- No two queens attack each other.

Time: $O(N!)$

Backtracking explores all valid placements.

Solve N-Queens (Skeleton)

```
static void solve(int row, char[][] board) {  
    if (row == board.length) {  
        printBoard(board);  
        return;  
    }  
  
    for (int col = 0; col < board.length; col++) {  
        if (isSafe(board, row, col)) {  
            board[row][col] = 'Q';  
            solve(row + 1, board);  
            board[row][col] = '.';  
        }  
    }  
}
```




Backtracking Pattern

1. Choose
2. Explore
3. Unchoose

Always undo changes.



Real World Use Case — Sudoku Solver

Try number → check valid → recurse → undo if invalid.

Used in:

- Puzzle solvers
- Game engines
- AI constraint solving



Real World Use Case — Path Finding

Maze solving:

- Explore path
- Mark visited
- Backtrack when dead-end



Complexity Summary

Problem	Time
Factorial	$O(n)$
Fibonacci	$O(2^n)$
Subsets	$O(2^n)$
Permutations	$O(n!)$
N-Queens	$O(N!)$

Backtracking often exponential.



Pruning Optimization

Avoid unnecessary branches.

Example:

If partial sum $>$ target $\rightarrow$ stop exploring.

Reduces search space.



Common Mistakes

- Missing base case
- Forgetting to backtrack
- Modifying shared state incorrectly
- Not visualizing recursion tree
- Ignoring exponential growth



Interview Discussion Points

- Difference between recursion and backtracking?
- Why backtracking exponential?
- How to prune effectively?
- How to convert recursion to iteration?
- How to estimate recursion complexity?



Mathematical Thinking

Recurrence relation example:

$$T(n) = 2T(n-1) + O(1)$$

Solves to:

$$O(2^n)$$

Understanding recurrence helps analyze recursion.



Summary

Recursion builds:

- Divide & conquer thinking
- Tree-based reasoning
- Mathematical modeling

Backtracking builds:

- State exploration
- Constraint solving
- Optimization thinking

Master:

- Base case logic
- Recursion tree visualization
- Backtracking template